

Architecture Design Report

This document presents the architectural design of the system, including its context, logical structure, runtime behavior, and deployment.

1. System Requirements

System Name: ResuFit: Precision CV Matching System

Architecture Style: Layered (N-Tier)

Functional Requirements

- The system must analyze gestures and reflect the corresponding action on the projected surface.
- The system will include gesture recognition.
- The system will include face recognition.
- The system will include hand tracking.
- The system will include single-hand interaction.
- The system will include customizing the actions for specific gestures.
- The system must enable real-time collaboration by supporting interactions on the projected surface.
- The system must have low-latency.
- The system must work effectively in different places.
- The system must be able to detect the user's face using the YOLOv5 model.
- The system must use contours detection to detect the borders of objects.
- The system must use Gaussian blur to mute the image noise.
- The system must use Background Subtraction (BS) to separate the moving parts.
- The system must use Histogram Equalization for image contrast optimization.
- The system must use threading for concurrent execution of tasks within a single process.
- The system must use multiprocessing for parallel execution of tasks across multiple processes.

Non-Functional Requirements

- The system must have a competitive price.

Description: The system must have a competitive price. After making a competitive analysis we figured out that the system must be able to work in real-time.

- The system must be able to work in real-time.

Description: The system must be able to work in real-time.

- The system must have low-latency.

Description: The system must have low-latency.

- The system must be able to handle multiple users.

Description: The system must be able to handle multiple users.

- The system must be able to handle different types of gestures.

Description: The system must be able to handle different types of gestures.

- The system must be able to handle different types of environments.

Description: The system must be able to handle different types of environments.

- The system must be able to handle different types of hardware.

Description: The system must be able to handle different types of hardware.

- The system must be able to handle different types of software.

Description: The system must be able to handle different types of software.

2. Formal Architecture Specification (ACME ADL)

The following section provides a formal architectural specification using ACME ADL, capturing components, con-

```
System ResuFit:PrecisionCVMatchingSystem {
  Property view : String = "runtime";
  Property architectureStyle : String = "Layered (N-Tier)";
  Property source : String = "AI";
  Component GestureAnalysis = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Analyze gestures and reflect the corresponding action on the";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component GestureRecognition = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Include gesture recognition.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component FaceRecognition = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Include face recognition.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component HandTracking = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Include hand tracking.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
}
```

```

Component Single-HandInteraction = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Include single-hand interaction.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component CustomizableGestures = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Customize the actions for specific gestures.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component Real-TimeCollaboration = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Enable real-time collaboration by supporting interactions on";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component Low-Latency = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Have low-latency.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component Place-Agnostic = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Work effectively in different places.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component FaceDetection = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Detect the user's face using the YOLOv5 model.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component ContourDetection = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Use contours detection to detect the borders of objects.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component GaussianBlur = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Use Gaussian blur to mute the image noise.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component BackgroundSubtraction = new Component {
    Port in;

```

```

    Port out;
    Property responsibility : String = "Use Background Subtraction (BS) to separate the moving parts";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component HistogramEqualization = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Use Histogram Equalization for image contrast optimization.";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component Threading = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Use threading for concurrent execution of tasks within a single process";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Component Multiprocessing = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Use multiprocessing for parallel execution of tasks across multiple processes";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
}
Attachment GestureAnalysis.out to GestureRecognition.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment GestureRecognition.out to FaceRecognition.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment FaceRecognition.out to HandTracking.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment HandTracking.out to Single-HandInteraction.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment Single-HandInteraction.out to CustomizableGestures.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment CustomizableGestures.out to Real-TimeCollaboration.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Attachment Real-TimeCollaboration.out to Low-Latency.in {
    Property delivery : String = "at-least-once";

```

```

    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment Low-Latency.out to Place-Agnostic.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment FaceDetection.out to ContourDetection.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment ContourDetection.out to GaussianBlur.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment GaussianBlur.out to BackgroundSubtraction.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment BackgroundSubtraction.out to HistogramEqualization.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment HistogramEqualization.out to Threading.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = true;
    Property backpressure : String = "bounded";
}
Attachment Threading.out to Multiprocessing.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
}
Property resilience_pattern : String = "Retry";
Property delivery : String = "at-most-once";
Property ordering : Boolean = true;
Property authentication : String = "centralized";
Property authorization : String = "RBAC";
}

```

3. Context View



